`thread_cond_any` pointer *once you're inside the* `wait()` *call* but not before. This is exactly what you were after (but couldn't manage) with `std::condition_variable`. Once `wait()` has finished waiting (either because it was notified or because of a spurious wake), it will call your `lock()` function, which again acquires the lock on the internal `set_clear_mutex` and the lock on the `Lockable` object ❹. You can now check again for interruptions that happened during the `wait()` call before clearing the `thread_cond_any` pointer in your `custom_lock` destructor ❺, where you also unlock the `set_clear_mutex`.

9.2.5. Interrupting other blocking calls

That rounds up interrupting condition variable waits, but what about other blocking waits: mutex locks, waiting for futures, and the like? In general you have to go for the timeout option you used for `std::condition_variable` because there's no way to interrupt the wait short of actually fulfilling the condition being waited for, without access to the internals of the mutex or future. But with those other things you do know what you're waiting for, so you can loop within the `interruptible_wait()` function. As an example, here's an overload of `interruptible_wait()` for a `std::future<>`:

```
template<typename T>
void interruptible_wait(std::future<T>& uf)
{
   while(!this_thread_interrupt_flag.is_set())
   {
       if(uf.wait_for(lk,std::chrono::milliseconds(1)==
           std::future_status::ready)
           break;
   }
   interruption_point();
}
```

This waits until either the interrupt flag is set or the future is ready but does a blocking wait on the future for 1 ms at a time. This means that on average it will be around 0.5 ms before an interrupt request is acknowledged, assuming a high-resolution clock. The `wait_for` will typically wait at least a whole clock tick, so if your clock ticks every 15 ms, you'll end up